`ArrayBlockingQueue` is a **thread-safe, bounded, FIFO queue** from Java's `java.util.concurrent` package. It's built on a **fixed-size array** and is commonly used in **producer–consumer** scenarios where you want *backpressure* instead of unbounded memory growth.

Mental model (what it *is*)

Think of it as:

> 🧺 A queue with a **fixed number of slots**
> ➕ Producers **block** when it's full
> ➖ Consumers **block** when it's empty
> 🔒 All operations are thread-safe

Key properties

Property	Description
Bounded	Capacity is fixed at creation time
FIFO	Elements come out in the same order they went in
Blocking	Threads can block on `put()` / `take()`
Array-backed	Uses a circular array internally
Optional fairness	Can use fair locking (FIFO thread access)

How it works internally

1. Fixed-size circular array

- Backed by an `Object[]`
- Two indexes:
  - `putIndex` → where the next element goes
  - `takeIndex` → where the next element comes from
- Indexes **wrap around** when they reach the end (circular buffer)

2. Single lock + conditions

Internally it uses:

- **One** `ReentrantLock`
- Two `Condition`s:
  - `notEmpty` → consumers wait here
  - `notFull` → producers wait here

This ensures **strong ordering guarantees** and simplicity.

What happens during operations

`put(E e)`

1. Acquire the lock
2. If queue is **full** → wait on `notFull`
3. Insert element at `putIndex`
4. Advance `putIndex`
5. Signal `notEmpty`
6. Release lock

⛔ Blocks indefinitely if full

`take()`

1. Acquire the lock
2. If queue is **empty** → wait on `notEmpty`
3. Remove element at `takeIndex`
4. Advance `takeIndex`
5. Signal `notFull`
6. Release lock

⛔ Blocks indefinitely if empty

Non-blocking & timed variants

- `offer(e)` → returns `false` if full

- `poll()` → returns `null` if empty
- `offer(e, timeout, unit)`
- `poll(timeout, unit)`

Fairness mode

```
BlockingQueue<Integer> q =
    new ArrayBlockingQueue<>(100, true);
```

- `fair = true`
  - Threads acquire the lock in **FIFO order**
  - Prevents starvation
  - ❌ Slightly slower due to scheduling overhead
- `fair = false` (default)
  - Higher throughput
  - Possible (but rare) starvation

When should you use ArrayBlockingQueue?

✅ Good choice when:

- You want **strict capacity limits**
- Predictable memory usage matters
- You want **simple, stable behavior**
- You have moderate contention

🚫 Not ideal when:

- Capacity needs to grow dynamically
- Very high contention → `LinkedBlockingQueue` may scale better
- You need lock-free behavior → look at `ConcurrentLinkedQueue`

Quick comparison

Queue	Capacity	Locks	Typical Use
`ArrayBlockingQueue`	Fixed	1	Backpressure, bounded systems
`LinkedBlockingQueue`	Optional	2	Higher throughput
`SynchronousQueue`	0	Hand-off	Direct producer → consumer
`PriorityBlockingQueue`	Unbounded	1	Priority-based tasks

Tiny example

```
BlockingQueue<String> queue = new ArrayBlockingQueue<>(2);

new Thread(() -> {
    try {
        queue.put("A");
        queue.put("B");
        queue.put("C"); // blocks here
    } catch (InterruptedException ignored) {}
}).start();

new Thread(() -> {
    try {
        Thread.sleep(1000);
        System.out.println(queue.take()); // unblocks producer
    } catch (Exception ignored) {}
}).start();
```

If you want, I can:

- Compare **ArrayBlockingQueue vs LinkedBlockingQueue** in depth
- Explain **memory visibility & happens-before guarantees**
- Walk through **real interview questions** around blocking queues